

OptiShift

Technical and Functional Specification

System Type: Data-driven SaaS ERP for hospitality workforce planning and operations

Prepared By: Mohammad

Technical Mentor: Aliakbar

Date: February 21, 2026

Status: Baseline specification for implementation

Contents

1	Document Control	1
2	Executive Summary	1
3	Strategic Objective and Program Alignment	1
4	Purpose, Scope, and Success Criteria	2
4.1	Purpose	2
4.2	In Scope	2
4.3	Optional Extension Scope (Time-Permitting)	2
4.4	Out of Scope (Current Release)	2
4.5	Success Criteria	2
5	Stakeholders and Roles	3
6	Assumptions and Constraints	3
7	System Context and Architecture	4
7.1	High-Level Architecture	4
7.2	Primary Data Flow	4
7.3	Optional Data Flow: Edge Telemetry (Time-Permitting)	4
8	Detailed Technology Stack	4
8.1	Platform Layers and Technologies	4
8.2	Technology Selection Notes	6
9	Hard Concepts Explained	6
9.1	Constraint Satisfaction Problem (CSP) and CP-SAT	6
9.2	Genetic Algorithm for Schedule Optimization	6
9.3	Time-Series Forecasting and Model Drift	6
9.4	Offline-First Synchronization	6
9.5	Digital Twin State Consistency	7
9.6	JWT and RBAC Together	7
9.7	MQTT, QoS, and Edge Buffering (Optional Extension)	7
10	Functional Requirements	7
10.1	Module 1: ERP and Legal Compliance Engine	7
10.2	Module 2: Predictive AI and Auto-Scheduling	7
10.3	Module 3: 3D Digital Twin and Spatial Mapping	8

10.4 Module 4: Offline-First Mobility	8
10.5 Module 5 (Optional, Time-Permitting): Network and Connected Objects	9
11 Non-Functional Requirements	9
12 Data Model Baseline	10
12.1 Core Entities	11
12.2 Schema Constraints	11
13 API Contract Baseline	12
14 Security and Compliance Baseline	12
15 Testing and Quality Gates	13
15.1 Test Layers	13
15.2 CI/CD Pipeline	13
16 Delivery Roadmap	14
16.1 Gantt Chart (12 Week Execution Plan)	14
17 Risk Register	15
18 Acceptance and Sign-Off Checklist	15
19 Traceability Matrix (Condensed)	16

1 Document Control

Document ID	OPTI-SPEC-004
Date	February 21, 2026
Owner	Lead Architect and Developer
Audience	Engineering, product, mentors, reviewers
Classification	Internal project specification

2 Executive Summary

OptiShift is a hospitality-focused ERP platform for legally compliant scheduling, forecasting-driven staffing optimization, and spatially aware operations management. The platform combines:

- deterministic labor-law enforcement for mission-critical scheduling constraints,
- machine-learning occupancy forecasting and constraint-based auto-scheduling,
- WebGL-based 3D operational visualization,
- offline-capable employee mobility features through PWA patterns,
- optional network and connected-object extension (time-permitting) for real-time operational telemetry.

This document transforms conceptual design into implementation-ready specifications with measurable outcomes and verification methods.

3 Strategic Objective and Program Alignment

Based on the project framing in D0C-20260130-WA0000. (3).pdf, OptiShift is positioned as a senior-junior level capstone that demonstrates both engineering rigor and business maturity for competitive Master’s admission.

- Domain legitimacy: project scope is grounded in real hospitality operations and scheduling pain points.
- Management and business value: process optimization, workforce cost control, and role-based enterprise workflows.
- Data and systems value: forecasting and optimization logic to model real-world complexity.
- Reliability and mobility value: legal safety checks, robust architecture, and offline-capable employee experience.
- Optional networking value (time-permitting): edge-to-cloud telemetry flow through secure message transport and device identity controls.

Target Track Focus	OptiShift Evidence
Management and Business IT	RBAC, scheduling workflows, dashboard-driven staffing decisions, and cost-aware planning.
Data and Complex Systems	Occupancy forecasting, rule-constrained optimization, and measurable model-driven recommendations.
Health and Technical Reliability	Safety-oriented rest-rule enforcement, PWA offline continuity, and auditable operational control.
Network, Internet, and Connected Systems (optional)	Optional IoT extension can demonstrate MQTT event ingestion, edge buffering, and observability over distributed connectivity.

4 Purpose, Scope, and Success Criteria

4.1 Purpose

Define system behavior, architecture, interfaces, and quality targets required to implement OptiShift reliably in real-world hospitality operations.

4.2 In Scope

- Workforce scheduling and legal compliance controls.
- Occupancy forecasting and staffing recommendation workflows.
- Visual floor-state rendering and employee assignment mapping.
- Employee-facing offline shift visibility and synchronization.
- CI/CD quality gates and release roadmap.

4.3 Optional Extension Scope (Time-Permitting)

- Optional network and connected-object track for ingesting occupancy/device telemetry from edge gateways.
- MQTT-based event pipeline with secure device identity and replay-safe ingestion.
- This extension is explicitly non-blocking for baseline release and will be implemented only if schedule allows.

4.4 Out of Scope (Current Release)

- Payroll disbursement to banks.
- Biometric attendance hardware integration.
- Multi-tenant white-label custom branding.

4.5 Success Criteria

- All mandatory legal constraints enforced at API layer with auditable logs.

- Draft schedules generated within target runtime and accepted by managers.
- Forecast service produces usable staffing recommendations by department.
- Employee users can read schedules while offline and sync updates safely.
- Optional extension success (only if activated): authenticated telemetry events are ingested and reflected in dashboards within target latency.

5 Stakeholders and Roles

Role	Primary Responsibility
Product Owner / Operations Lead	Prioritizes workflows, approves sprint deliverables.
Engineering Lead	Owns architecture, coding standards, and release quality gates.
Backend Engineer	Builds APIs, compliance engine, persistence, and integrations.
ML Engineer	Builds forecasting pipeline, model evaluation, and inference service.
Frontend Engineer	Delivers manager dashboard, employee PWA, and 3D interface.
QA Engineer	Designs functional, regression, and end-to-end test plans.

6 Assumptions and Constraints

Operating Assumptions

- Primary deployment target is one hospitality group with standardized shift policies.
- Baseline legal rule: minimum rest period between shifts is 11 hours.
- Historical occupancy data is available for model training.
- Teams operate on weekly scheduling cycles with manager approvals.
- Optional extension assumption: if time allows, selected properties can expose basic edge telemetry through a gateway layer.

Key Constraints

- Must run as containerized microservices.
- PostgreSQL is the system of record for operational data.
- Authentication and authorization must be JWT and role-based.
- All critical decisions must be traceable in logs for audit review.
- Optional network extension must not delay baseline release gate (Weeks 1–12).

7 System Context and Architecture

7.1 High-Level Architecture

OptiShift uses a microservice architecture with three core runtime services:

- **optishift-api** (Node.js/Express): business rules, RBAC, scheduling, and orchestration.
- **optishift-ai** (Python/FastAPI): forecasting and staffing recommendations.
- **optishift-db** (PostgreSQL 15): transactional persistence and reporting queries.

Optional time-permitting extension services:

- **optishift-ingest** (FastAPI or NestJS worker): receives telemetry events, validates device identity, and normalizes payloads.
- **message broker** (MQTT): transports edge telemetry with topic-based routing.

7.2 Primary Data Flow

1. Manager submits shift draft via frontend.
2. API validates identity, permissions, and legal constraints.
3. API persists approved assignments and emits audit records.
4. AI service consumes occupancy history and returns staffing recommendations.
5. 3D view reads assignment state and maps users to spatial coordinates.

7.3 Optional Data Flow: Edge Telemetry (Time-Permitting)

1. Edge gateway publishes occupancy/device events to MQTT topics.
2. Ingest service authenticates source, validates schema, and deduplicates event IDs.
3. Valid telemetry is persisted for analytics and surfaced to operations dashboards.

8 Detailed Technology Stack

8.1 Platform Layers and Technologies

Layer	Technology	Implementation Detail and Why It Is Used
Web frontend	React + TypeScript + Tailwind CSS	Component-driven UI for manager dashboard and employee portal. TypeScript gives stricter contracts and fewer runtime bugs. Tailwind speeds up consistent design tokens.

Layer	Technology	Implementation Detail and Why It Is Used
3D visualization	Three.js + React Three Fiber	Renders floor plans and employee assignment nodes. React Three Fiber keeps the 3D scene manageable in the same component system as the rest of the frontend.
Offline mobile UX	PWA (Service Worker + IndexedDB)	Enables shift visibility and deferred actions without internet. IndexedDB stores cached schedule snapshots and queued user intents for later synchronization.
API backend	Node.js + NestJS (or Express modular architecture)	Handles RBAC, legal validation, scheduling workflows, and orchestration between persistence and AI services. NestJS-style modular structure keeps domain boundaries explicit.
AI microservice	Python + FastAPI + pandas/scikit-learn	Hosts forecast endpoints and optimization logic. FastAPI gives typed contracts and high-performance async serving for model inference.
Optimization engine	OR-Tools CP-SAT + Genetic Algorithm strategy	CP-SAT solves hard constraints (law and staffing minimums). Genetic search explores near-optimal schedules when objective complexity grows.
Primary database	PostgreSQL 15	ACID transactions, strong relational constraints, and reliable query performance for shift, department, and audit domains.
Cache and queues	Redis + background job workers	Improves read performance for repeated dashboard queries and runs forecast/scheduler tasks asynchronously without blocking API requests.
CI/CD and runtime	Docker + Docker Compose + GitHub Actions	Reproducible local/staging environments, automated tests, linting, image builds, and controlled deployment pipeline gates.
Testing stack	Jest, Supertest, Playwright, pytest	Unit + integration + E2E + model-service checks provide coverage across legal rules, API behavior, UI flow, and forecasting contracts.
Observability	OpenTelemetry, structured logs, Prometheus/Grafana	Correlates requests across services, tracks latency and errors, and gives actionable dashboards for reliability and incident response.

Layer	Technology	Implementation Detail and Why It Is Used
Optional network extension	MQTT broker (Mosquitto or EMQX), edge gateway agent, protobuf/JSON schema validation	Time-permitting track for connected-object ingestion. Broker decouples producers and consumers; gateway buffers intermittent connectivity; schema validation protects data contracts.

8.2 Technology Selection Notes

- The architecture is intentionally polyglot: JavaScript/TypeScript for product APIs and Python for ML-heavy workloads.
- Strict API contracts between services are required to keep independent deployment possible.
- Every critical workflow (schedule write, legal rejection, forecast job) must emit traceable telemetry.
- Optional network extension is a controlled add-on: it is enabled only if baseline quality gates are already green.

9 Hard Concepts Explained

9.1 Constraint Satisfaction Problem (CSP) and CP-SAT

A CSP means finding a valid schedule while respecting non-negotiable rules. In OptiShift, examples include minimum rest periods and minimum staffing by department. CP-SAT is used to guarantee these hard rules are never violated before optimization goals are considered.

9.2 Genetic Algorithm for Schedule Optimization

A genetic algorithm starts from many candidate schedules, scores them, and repeatedly mutates/recombines them. Over iterations, weak schedules are discarded and stronger ones survive. In OptiShift, this is useful when there are many competing soft objectives such as fairness, cost, and preference balancing.

9.3 Time-Series Forecasting and Model Drift

Time-series forecasting estimates future occupancy from historical booking behavior. Model drift occurs when real-world behavior changes and the old model becomes less accurate. OptiShift should monitor prediction error over time and retrain when accuracy falls below the release threshold.

9.4 Offline-First Synchronization

Offline-first means users can continue core actions with unstable connectivity. When connection returns, local queued actions sync to the server. Conflict policies are required because server data may change while the client is offline.

9.5 Digital Twin State Consistency

The digital twin is a visual mirror of operational state, not the source of truth. PostgreSQL remains authoritative, while the 3D layer reflects API state with low delay. This avoids visual drift and keeps legal/operational decisions anchored to auditable backend records.

9.6 JWT and RBAC Together

JWT proves identity and carries claims; RBAC uses those claims plus server-side rules to authorize actions. In OptiShift, a valid token alone is not enough: department scope and role permissions must still pass.

9.7 MQTT, QoS, and Edge Buffering (Optional Extension)

MQTT is a lightweight publish-subscribe protocol useful for unstable connectivity and many producers. For OptiShift's optional extension, QoS levels and gateway-side buffering reduce event loss during short outages. Events are accepted only when they pass device authentication and schema validation.

10 Functional Requirements

10.1 Module 1: ERP and Legal Compliance Engine

ID	Requirement	Acceptance Criterion
FR-LEG-001	System shall reject shift assignments when $\Delta t < 11$ hours between prior shift end and new shift start.	API returns HTTP 409 with reason code <code>REST_WINDOW_VIOLATION</code> . Unit and integration tests pass.
FR-LEG-002	System shall enforce RBAC for schedule operations by department scope.	Unauthorized cross-department update attempt returns HTTP 403 and audit event is logged.
FR-LEG-003	System shall compute labor cost totals including overtime and night premium factors.	Payroll analytics endpoint returns total cost and factor breakdown for a selected date range.
FR-LEG-004	All schedule create/update/delete events shall produce immutable audit records.	Audit table contains actor, timestamp, change summary, and request correlation ID.

10.2 Module 2: Predictive AI and Auto-Scheduling

ID	Requirement	Acceptance Criterion
FR-AI-001	Forecasting service shall predict occupancy for at least 7 future days by department-relevant granularity.	Forecast endpoint returns dated predictions and confidence metadata for each horizon day.
FR-AI-002	Auto-scheduler shall generate draft schedules minimizing labor cost while satisfying legal and minimum staffing constraints.	Generated schedule includes no hard-constraint violations and objective cost value.
FR-AI-003	Forecast and scheduler runs shall store versioned execution metadata.	Metadata includes model version, data window, runtime, and execution status.

10.3 Module 3: 3D Digital Twin and Spatial Mapping

ID	Requirement	Acceptance Criterion
FR-3D-001	Client shall render floor plan in WebGL using Three.js or equivalent scene graph engine.	Scene initializes with configured floor meshes and camera controls on supported browsers.
FR-3D-002	Employee assignments shall be mapped to coordinates and rendered as interactive nodes.	Clicking node opens assignment details (user, role, floor, shift window).
FR-3D-003	Spatial state refresh shall be near real-time.	Assignment updates visible in the twin within 5 seconds of API state change in staging tests.

10.4 Module 4: Offline-First Mobility

ID	Requirement	Acceptance Criterion
FR-MOB-001	Employee portal shall support schedule read access when offline.	With network disabled, cached schedule remains viewable with last-sync timestamp.
FR-MOB-002	Client shall queue supported user actions while offline and retry on reconnect.	Queued actions synchronize successfully or surface conflict resolution prompt.

ID	Requirement	Acceptance Criterion
FR-MOB-003	Service Worker and IndexedDB storage shall implement safe cache invalidation.	Versioned cache policy clears obsolete assets without corrupting local shift data.

10.5 Module 5 (Optional, Time-Permitting): Network and Connected Objects

ID	Requirement	Acceptance Criterion
FR-NET-001	If optional extension is activated, system shall ingest MQTT telemetry from approved edge gateways using authenticated clients.	In staging, authenticated gateway events are accepted and unauthorized clients are rejected.
FR-NET-002	Ingest pipeline shall perform schema validation and event ID deduplication before persistence.	Invalid payloads return validation error and duplicates do not create multiple records.
FR-NET-003	Telemetry dashboard view shall display latest connected-source status and event freshness.	Dashboard shows source heartbeat timestamp and latest event age for monitored topics.

11 Non-Functional Requirements

ID	Category	Target	Verification
NFR-AVL-001	Availability	$\geq 99.5\%$ monthly uptime (excluding planned maintenance)	Synthetic uptime checks and incident reports.
NFR-PERF-001	API Performance	p95 response time < 300 ms for core scheduling APIs at baseline load	Load tests in CI and staging benchmarks.
NFR-PERF-002	Scheduler Runtime	Draft schedule generation for one week completes in < 90 seconds for baseline property size	Performance test dataset and timed job telemetry.

ID	Category	Target	Verification
NFR-SEC-001	Authentication	JWT access tokens with expiry and refresh policy; role claims validated server-side	Security integration tests and token misuse tests.
NFR-SEC-002	Data Protection	Password hashing, TLS in transit, least-privilege DB access	Security checklist and deployment config audit.
NFR-OBS-001	Observability	Structured logs, trace IDs, and service metrics for all critical paths	Log sampling and dashboard review during test runs.
NFR-MNT-001	Maintainability	Service boundaries documented with API contracts and migration history	Architecture review and docs completeness check.
NFR-NET-001	Optional network extension	If enabled, telemetry ingestion p95 end-to-dashboard latency < 5 seconds under staging baseline load	Telemetry load test and dashboard freshness checks (executed only when optional track is activated).

12 Data Model Baseline

12.1 Core Entities

Table	Primary Key	Key Fields and Relationships
Users	user_id (UUID)	role_id (FK), dept_id (FK), base_salary, contract_hours, password_hash
Shifts	shift_id (UUID)	user_id (FK), dept_id (FK), start_time, end_time, status
Departments	dept_id (UUID)	name, min_staff_required, cost_center_code
OccupancyLog	log_id (UUID)	date, rooms_booked, forecasted_occupancy, model_version
AuditLog	audit_id (UUID)	actor_id, action_type, entity_type, entity_id, request_id, timestamp

Schema Readability Note

When embedding the ER diagram, use high-contrast labels, minimum 10pt node text, and avoid low-opacity connectors so the schema remains readable in print and on-screen.

12.2 Schema Constraints

- Index Shifts(user_id, start_time) for fast rest-window checks.
- Enforce UTC timestamp storage for shift boundaries.
- Use foreign key constraints to prevent orphaned schedule records.
- Keep schema migrations versioned and reversible.

13 API Contract Baseline

Method	Path	Auth	Behavior
POST	/api/v1/auth/login	Public	Returns JWT access and refresh tokens.
POST	/api/v1/shifts	Manager+	Creates shift after legal and staffing validation checks.
PATCH	/api/v1/shifts/{id}	Manager+	Updates shift and revalidates legal constraints.
GET	/api/v1/schedules/{deptId}	Department scope	Returns date-bounded schedule view by department.
POST	/api/v1/forecast/run	Manager+	Triggers occupancy forecasting job and returns run ID.
GET	/api/v1/twin/state	Authenticated	Returns coordinate-mapped assignment snapshot.
POST	/api/v1/iot/events (optional)	Device token / mTLS gateway	Receives telemetry payloads when optional network extension is activated.
GET	/api/v1/iot/status (optional)	Manager+	Returns source health and latest event timestamps when optional extension is active.

14 Security and Compliance Baseline

- RBAC policies defined per endpoint and department ownership.
- JWT signing keys rotated by operational policy.
- Sensitive fields encrypted at rest where applicable.
- Audit logs retained based on legal and operational retention windows.
- Legal rule configuration externalized for future jurisdiction expansion.
- Optional network extension security: device identity verification (token or mTLS), topic-level access control, and replay protection for event IDs.

15 Testing and Quality Gates

15.1 Test Layers

- Unit tests for legal rule calculations and utility functions.
- Integration tests for API + DB behavior (including 409 and 403 paths).
- ML validation tests for forecast quality and data contract checks.
- End-to-end tests for manager workflow and employee offline workflow.
- Optional extension tests (only if activated): MQTT ingest auth tests, schema validation tests, and telemetry freshness checks.

15.2 CI/CD Pipeline

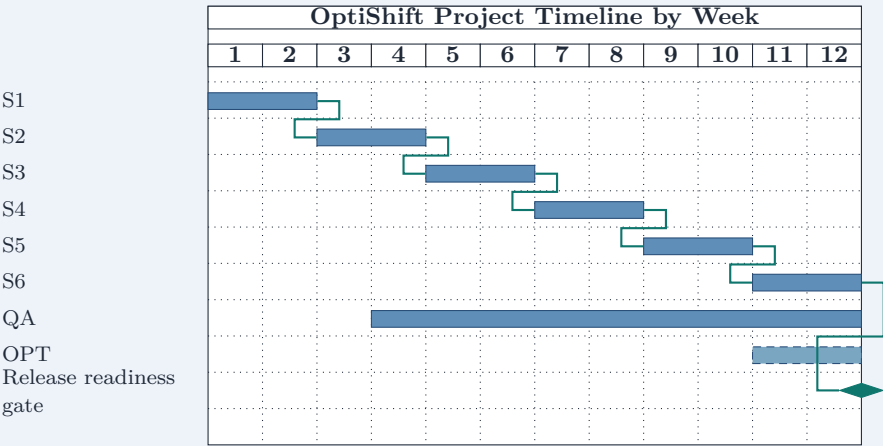
1. Lint and static checks.
2. Unit and integration test suites.
3. Build Docker images for API and AI services.
4. Deploy to staging and run smoke tests.
5. Gate production release on explicit approval.

16 Delivery Roadmap

Sprint	Start	End	Theme	Key Deliverables
S1	Week 1	Week 2	Architecture and Infrastructure	Schema approval, API scaffold, CI baseline.
S2	Week 3	Week 4	Security and Core API	JWT auth, RBAC checks, shift CRUD, legal validation.
S3	Week 5	Week 6	Frontend and PWA	Manager dashboard, employee portal, offline sync queue.
S4	Week 7	Week 8	AI Integration	Forecast pipeline, baseline model, recommendation API.
S5	Week 9	Week 10	3D Digital Twin	WebGL renderer, coordinate map, assignment interaction.
S6	Week 11	Week 12	Hardening and Release	End-to-end stabilization, release hardening, final docs.
OPT	Week 11	Week 12	Optional Network Extension (if time)	MQTT ingest PoC, gateway auth, telemetry dashboard status cards.

16.1 Gantt Chart (12 Week Execution Plan)

Roadmap note: Bars labeled as optional are executed only if baseline sprints (S1–S6) remain on track and quality gates are green. Code key: S1–S6 = baseline sprint sequence; QA = cross-cutting quality assurance/UAT stream (Weeks 4–12); OPT = optional network extension.



17 Risk Register

Risk	Severity	Mitigation
Poor data quality for forecasting	High	Add data validation pipeline and fallback staffing heuristics.
Complex legal rule changes by region	High	Externalize rule configuration and implement rule versioning.
3D twin performance on low-end devices	Medium	Add quality presets and progressive rendering strategy.
Offline sync conflicts	Medium	Implement deterministic conflict policy and user prompts.
Scope creep across modules	Medium	Enforce requirement traceability and sprint acceptance gates.
Optional network extension delays baseline	Medium	Time-box optional work, start only after baseline health checks, and permit deferral without blocking release.

18 Acceptance and Sign-Off Checklist

- All mandatory FR and NFR entries mapped to tests and verification evidence.
- No open high-severity security findings in staging environment.
- UAT sign-off from operations workflow owner.
- Deployment runbook validated with rollback drill.
- Documentation package complete (API, architecture, operations).

19 Traceability Matrix (Condensed)

Feature Area	Requirements	Primary Verification
Legal compliance engine	FR-LEG-001..004	Unit + integration tests, audit-log assertions
AI scheduling	FR-AI-001..003	Model validation tests, scheduler constraint checks
3D digital twin	FR-3D-001..003	UI integration tests and latency checks
Offline mobility	FR-MOB-001..003	Offline E2E scenarios and sync conflict tests
Optional network extension	FR-NET-001..003, NFR-NET-001	Telemetry ingest tests, auth checks, and freshness latency validation (if optional track is activated)
Platform quality	NFR suite (availability, performance, security, observability, maintainability)	Load tests, security checks, observability reviews